

Universidade de São Paulo
Instituto de Ciências Matemáticas e de Computação
SSC0605 - Teoria da computação e compiladores



Relatório

Primeiro Trabalho Compiladores

Alunos:

Aruan Bretas de Oliveira Filho (12609731)

Gabriel Garcia Ferreira (13677160)

Johnatas Luiz dos Santos (13676388)

Docente:

Thiago A. S. Pardo

Abril
2025

Universidade de São Paulo
Instituto de Ciências Matemáticas e de Computação
SSC0605 - Teoria da Computação e Compiladores

Relatório

Primeiro Trabalho Teoria da Computação e Compiladores

Alunos:

Aruan Bretas de Oliveira Filho (12609731)

Gabriel Garcia Ferreira (13677160)

Johnatas Luiz dos Santos (13676388)

Docente:

Thiago A. S. Pardo

Abril
2025

Sumário

1	Introdução	1
2	Implementação	2
2.1	Máquinas de estados feitas	2
2.2	Implementação dos códigos	3
3	Escolhas adotadas	7
3.1	Organização dos automatos	7
3.2	Tratamento de Tokens específicos	9
3.3	Controle de erros	10
3.4	Funções auxiliares	10
4	Conclusão	14
4.1	Integração dos automatos	14
4.2	Desafios Superados	14
4.3	Resultados alcançados	14

1 Introdução

No âmbito da disciplina de Teoria da Computação e Compiladores, a análise léxica constitui a primeira etapa no processo de compilação, ela é responsável por transformar um fluxo de caracteres brutos em unidades significativas, denominadas tokens. Esses tokens servem como base para as fases que vem em seguida, que são a análise sintática e semântica, fazendo com que o compilador compreenda e valide a estrutura do código fonte.

Este trabalho tem como objetivo desenvolver um analisador léxico para a linguagem PL/0, (figura1), uma linguagem de programação didática criada por Niklaus Wirth, conhecida por sua simplicidade e fortemente utilizada em estudos sobre compiladores. A gramática da PL/0, baseada em blocos estruturados, inclui declarações de constantes, variáveis, procedimentos e comandos como atribuições, condicionais (IF), laços (WHILE), e chamadas de procedimentos (CALL). Além disso, a linguagem suporta comentários entre chaves `/* */`, identificadores alfanuméricos (iniciados por letras) e números inteiros.

Gramática da linguagem PL/0
Fonte: Wirth, Niklaus. (1976). *Algorithms + Data Structures = Programs*. Prentice Hall, Inc.
(mais detalhes em <https://en.wikipedia.org/wiki/PL0>)

```
<programa> ::= <bloco> .
<bloco> ::= <declaracao> <comando>
<declaracao> ::= <constante> <variavel> <procedimento>
<constante> ::= CONST ident = numero <mais_const> ; | λ
<mais_const> ::= , ident = numero <mais_const> | λ
<variavel> ::= VAR ident <mais_var> ; | λ
<mais_var> ::= , ident <mais_var> | λ
<procedimento> ::= PROCEDURE ident ; <bloco> ; <procedimento> | λ
<comando> ::= ident := <expressao>
                | CALL ident
                | BEGIN <comando> <mais_cmd> END
                | IF <condicao> THEN <comando>
                | WHILE <condicao> DO <comando>
                | λ
<mais_cmd> ::= ; <comando> <mais_cmd> | λ
<expressao> ::= <operador_unario> <termo> <mais_termos>
<operador_unario> ::= - | + | λ
<termo> ::= <fator> <mais_fatores>
<mais_termos> ::= - <termo> <mais_termos> | + <termo> <mais_termos> | λ
<fator> ::= ident | numero | ( <expressao> )
<mais_fatores> ::= * <fator> <mais_fatores> | / <fator> <mais_fatores> | λ
<condicao> ::= ODD <expressao>
                | <expressao> <relacional> <expressao>
<relacional> ::= = | < > | <= | > | >=
```

Figura 1:

Para implementar o analisador léxico criamos primeiro autômatos finitos que modelam as regras léxicas da PL/0, para reconhecer as palavras reservadas (como CONST, VAR, PROCEDURE) os operadores (=, <>, <=, etc.), delimitadores e estruturas de comentários. Utilizamos a linguagem C descrita no relatório, sendo a função principal uma função que simula a interação entre o analisador léxico e sintático. O sistema processa um arquivo de entrada em txt e gera um arquivo de saída listando os pares token-classe, além de sinalizar erros léxicos também.

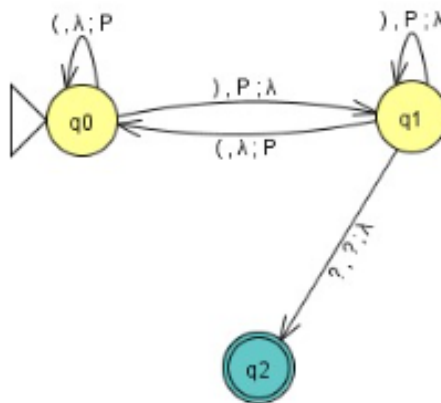


Figura 5: Parênteses

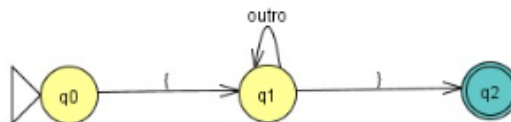


Figura 6: Comentários

2.2 Implementação dos códigos

- Vamos começar com a implementação dos identificadores e das palavras reservadas:

```

1 static void automatoIdentificador(const char* t,int err,int
2   len,int l){
3   // Erro em caso de caractere inválido ou cadeia muito
4   grande
5   if (err || len > 99) {
6     Token e = { .tipo = TOKEN_ERROR_LEXICO, .linha = l, .
7       status = 0 };
8     if (err) e.status |= 1;
9     if (len > 99) e.status |= 2;
10    strncpy(e.lexema, t, 99); e.lexema[99]=0;
11    addToken(e);
12    return;
13  }
14  // Palavra válida, verifica se é identificador ou palavra
15  reservada
16  Token r = { .linha = l, .status = 0 };
17  int ht = hash_buscar(&tabelaPalavrasReservadas, (string)t
18  ;

```

```

14     r.tipo = (ht >= 0) ? (TokenType)ht : TOKEN_IDENTIFIER;
15     strcpy(r.lexema, t);
16     addToken(r);
17 }

```

- Implementação do automato de números inteiros ou reais

```

1  static void automatoNumero(const char* t,int err,int len,int l
    ){
2      // erro de número mal formatado ou número muito longo
3      if (err || len > 99) {
4          Token e = { .tipo = TOKEN_ERROR_LEXICO, .linha = 1, .
              status = 0 };
5          if (err) e.status |= 1;
6          if (len > 99) e.status |= 2;
7          strncpy(e.lexema, t, 99); e.lexema[99]=0;
8          addToken(e);
9          return;
10     }
11     // numero válido
12     for(int i = 0; i < len; i++) {
13         if (!isdigit(t[i])) {
14             Token e = { .tipo = TOKEN_ERROR_LEXICO, .linha = 1
                , .status = 0 };
15             e.status |= 1;
16             strncpy(e.lexema, t, 99); e.lexema[99]=0;
17             addToken(e);
18             return;
19         }
20     }
21
22
23     Token r = { .tipo = TOKEN_NUMBER, .linha = 1, .status = 0
        };
24     strcpy(r.lexema, t);
25     addToken(r);
26 }

```

- Implementação do automato de parênteses

```

1  //identifica operador
2  int automatoSymbol(const char *c, char next, int l){
3      // tratando em caso de operador ser formado por 2 simbolos

```

```

4     if (c[0] == ':' && next == '=') {
5         Token r = { .tipo = TOKEN_ASSIGN, .linha = 1, .status
6                     = 0 };
7         strcpy(r.lexema, ":=");
8         addToken(r);
9         return 2;
10    }
11    if (c[0] == '<' && next == '=') {
12        Token r = { .tipo = TOKEN_LE, .linha = 1, .status = 0
13                    };
14        strcpy(r.lexema, "<=");
15        addToken(r);
16        return 2;
17    }
18    if (c[0] == '>' && next == '=') {
19        Token r = { .tipo = TOKEN_GE, .linha = 1, .status = 0
20                    };
21        strcpy(r.lexema, ">=");
22        addToken(r);
23        return 2;
24    }
25    if (c[0] == '<' && next == '>') {
26        Token r = { .tipo = TOKEN_NE, .linha = 1, .status = 0
27                    };
28        strcpy(r.lexema, "<>");
29        addToken(r);
30        return 2;
31    }
32
33    Token r = {0}; r.linha = 1; r.status = 0;
34    char s2[3] = { c[0], next? next : '\0', '\0' };
35    int ht = hash_buscar(&tabelaSimbolos, (string)s2);
36    if (ht < 0) {
37        s2[1] = '\0';
38        ht = hash_buscar(&tabelaSimbolos, (string)s2);
39    }
40    r.tipo = (ht >= 0) ? (TokenType)ht : TOKEN_ERROR_LEXICO;
41    strcpy(r.lexema, s2);
42
43    int tokelen = s2[1] ? 2 : 1;
44    // skip comma tokens

```



```

41     if (r.tipo != TOKEN_COMMA) {
42         addToken(r);
43     }
44     return tokelen;
45 }

```

- Implementação do automato de comentarios

```

1  int automatoComentario(const char* linha, int pointer, int
   num_linha) {
2      int start = pointer;
3      pointer++; // skip '{'
4      while (linha[pointer] != '\0' && linha[pointer] != '}') {
5          pointer++;
6      }
7      if (linha[pointer] == '}') {
8          pointer++;
9      } else {
10         int len = pointer - start;
11         int copy_len = (len > 99) ? 99 : len;
12         // strip trailing newline if present
13         if (copy_len > 0 && linha[start + copy_len] == '\n')
14             copy_len--;
15         Token e = { .tipo = TOKEN_ERROR_LEXICO, .linha =
16                     num_linha, .status = 1 };
17         strncpy(e.lexema, linha + start, copy_len);
18         e.lexema[copy_len] = '\0';
19         addToken(e);
20     }
21     return pointer - start;
22 }

```

3 Escolhas adotadas

3.1 Organização dos automatos

Utilizamos o Jflap para mapear os Automatos e em seguida poder codifica-los na linguagem C. Mapeamos um de cada vez, e utilizamos uma função main para identificar qual seria o automato utilizado para realizar a análise léxica:

```
1 void lexico(const char* linha, int num_linha){
2     // inicializa a tabela hash na primeira chamada
3     static int initialized = 0;
4     if (!initialized) {
5         inicializarTabelaReservadas();
6         initialized = 1;
7     }
8
9     int pointer = 0;
10    char caracter[2];
11
12    DBG_PRINT("\nAnalisando linha %d: '%s'\n", num_linha, linha);
13
14    while (linha[pointer] != '\0') {
15        caracter[0] = linha[pointer];
16        caracter[1] = '\0';
17
18        DBG_PRINT("Caractere atual: '%c' (posição %d)\n", caracter
19                [0], pointer);
20
21        if (isspace(caracter[0])) {
22            DBG_PRINT("Ignorando espaço\n");
23            pointer++;
24            continue;
25        }
26
27        // Tratar comentário {...}
28        if (caracter[0] == '{') {
29            DBG_PRINT("Caractere inicia comentário\n");
30            int avanco = automatoComentario(linha, pointer,
31                num_linha);
32            DBG_PRINT("Avançando %d posições (comentário)\n",
33                avanco);
34            pointer += avanco;
35            continue;
36        }
37    }
38 }
```

```

33     }
34     //trata os operadores
35     if (isSymbol(caracter)) {
36         DBG_PRINT("Caractere é um símbolo\n");
37         char next_char = '\0';
38         if (linha[pointer + 1] != '\0') {
39             next_char = linha[pointer + 1];
40         }
41         int avanco = automatoSymbol(caracter, next_char,
42                                     num_linha);
43         DBG_PRINT("Avançando %d posições\n", avanco);
44         pointer += avanco;
45         continue;
46     }
47     //trata identificador e palavra reservada
48     if (isalpha(caracter[0])) {
49         DBG_PRINT("Caractere é uma letra\n");
50         scanTermo(linha, &pointer, num_linha,
51                  automatoIdentificador);
52         continue;
53     }
54     // 4) idem para números
55     if (isdigit(caracter[0])) {
56         DBG_PRINT("Caractere é um dígito\n");
57         // scan integer (dots cause err)
58         int err_num, len_num = tamanhoNumero(linha, pointer, &
59                                               err_num);
60         char termo[100];
61         int sz = len_num < 99 ? len_num : 99;
62         strncpy(termo, linha + pointer, sz);
63         termo[sz] = '\0';
64         automatoNumero(termo, err_num, len_num, num_linha);
65         pointer += len_num;
66         continue;
67     }
68     // 5) caso não reconhecido
69     DBG_PRINT("Caractere não reconhecido: '%c'\n", caracter[0]);
70     ;
71     Token error = {0}; // Initialize struct

```

```

70     error.linha = num_linha;
71     error.status = 1; // Indicate error status
72     error.tipo = TOKEN_ERROR_LEXICO; // Set the error type
73     strcpy(error.lexema, character); // Store the problematic
        character
74     addToken(error);
75     pointer++; // Evitar loop infinito
76 }
77 DBG_PRINT("Fim da linha %d\n", num_linha);
78 }

```

3.2 Tratamento de Tokens específicos

Para tratar as palavras reservadas e tokens específicos nós utilizamos uma tabela, que está denotada no começo do código do arquivo lexico.c, e para melhor entendimento de como tudo está sendo armazenado, tem que ser lido o arquivo hash.c:

```

1 // Define global variables
2 typedef struct { const char *lex; TokenType tipo;} Reservado;
3 static Reservado tabelaReservados[] = {
4     {"CALL",TOKEN_CALL}, {"VAR",TOKEN_VAR}, {"BEGIN",TOKEN_BEGIN},
        {"END",TOKEN_END}, {"WHILE",TOKEN_WHILE}, {"CONST",
        TOKEN_CONST}, {"PROCEDURE",TOKEN_PROCEDURE}, {"ELSE",
        TOKEN_ELSE}, {"THEN",TOKEN_THEN}, {"IF",TOKEN_IF}, {"DO",
        TOKEN_DO}, {"FOR",TOKEN_FOR}, {NULL,0}
5 };
6
7 // restore comma to symbol list for delimiter purposes
8 static const char *simbolos[] = {
9     ";", ":", "+", "-", "*", "/", "(", ")", "=", ",", ">", "<", "."
10 };
11 static TokenType simbolosType[] = {
12     TOKEN_SEMICOLON, TOKEN_COLON, TOKEN_PLUS, TOKEN_MINUS,
13     TOKEN_MULT,      TOKEN_DIV,   TOKEN_LPAREN, TOKEN_RPAREN,
14     TOKEN_EQUAL,     TOKEN_COMMA, TOKEN_GT,    TOKEN_LT,
        TOKEN_DOT
15 };

```

3.3 Controle de erros

Como tratamos erros léxicos, no caso, na leitura de algum caractere que não faz parte nem do alfabeto aceito pela linguagem:

```
1      //caractere lido não possui nenhum estado ao qual é aceito
      para entrar em um automato
2      DBG_PRINT("Caractere não reconhecido: '%c'\n", character[0])
      ;
3      Token error = {0}; // Initialize struct
4      error.linha = num_linha;
5      error.status = 1; // Indicate error status
6      error.tipo = TOKEN_ERROR_LEXICO; // Set the error type
7      strcpy(error.lexema, character); // Store the problematic
      character
8      addToken(error);
9      pointer++; // Evitar loop infinito
```

3.4 Funções auxiliares

Usamos muitas funções para identificar quando se é um símbolo, um delimitador, uma palavra reservada, o que foi muito útil durante o processo e reutilizado em diversos momentos, trechos amostrado abaixo:

```
1      int isReservedWord(const char *palavra) {
2          return hash_buscar(&tabelaPalavrasReservadas, (string)
          palavra) >= 0;
3      }
4
5      // signature fix; drop TRUE/FALSE
6      int isSymbol(const char *s) {
7          return hash_buscar(&tabelaSimbolos, (string)s) >= 0;
8      }
9
10     static int isSymbolChar(char c) {
11         char tmp[2] = {c, '\0'};
12         return isSymbol(tmp);
13     }
14
15     int isDelimiter(char c){
16         return isSymbolChar(c) || isspace((unsigned char)c);
17     }
```

Além das funções para identificação de caractere, também temos as que scanneam os números e termos, identificando previamente possíveis erros em sua cadeia, como observa-se a seguir:

```
1 // generic scan
2 void scanTermo(const char *linha, int *ptr, int num_linha,
3               void (*classify)(const char*,int,int,int)) {
4     int err, len = tamanhoTermo(linha,*ptr,&err);
5     char termo[100];
6     int sz = len<99?len:99;
7     strncpy(termo, linha+*ptr, sz); termo[sz]=0;
8     classify(termo, err, len, num_linha);
9     *ptr += len;
10 }
11
12 // alterar tamanhoTermo para reportar erro de caracter inválido (
13 // underscore now invalid)
14 static int tamanhoTermo(const char* linha, int pos, int* erro) {
15     int len = 0;
16     *erro = 0;
17     while (linha[pos+len] != '\0' && !isDelimiter(linha[pos+len]))
18     {
19         char c = linha[pos+len];
20         if (!isalnum(c) && !isSymbolChar(c)) {
21             *erro = 1;
22         }
23         len++;
24     }
25     return len;
26 }
27
28 // new: scan full numeric term (allow '.' but flag as error)
29 static int tamanhoNumero(const char *linha, int pos, int *erro) {
30     int len = 0;
31     *erro = 0;
32     while (linha[pos+len] != '\0' && !isspace((unsigned char)linha[
33         pos+len])
34         && !isSymbolChar(linha[pos+len])) {
35         char c = linha[pos+len];
36         if (!isdigit(c)) {
37             *erro = 1; // flag float or invalid char
38         }
39     }
```

```

36         len++;
37     }
38     return len;
39 }

```

Ainda dentro do arquivo lexico.c, temos as funções que inicializa a tabela de reservados e uma que libera da memória, e por fim a que adiciona as cadeias lidas transformando elas em tokens inserindo-as em nossa estrutura de tokens lidos, como feito abaixo:

```

1     void inicializarTabelaReservadas() {
2         // Count the number of reserved words
3         int numPalavrasReservadas = 0;
4         while (tabelaReservados[numPalavrasReservadas].lex != NULL) {
5             numPalavrasReservadas++;
6         }
7
8         // Create a hash table with size about 2x the number of words (
           using prime number)
9         int tamanhoTabela = numPalavrasReservadas * 2 + 1;
10        hash_criar(&tabelaPalavrasReservadas, tamanhoTabela);
11
12        // Insere todas as palavras reservadas
13        for (int i = 0; i < numPalavrasReservadas; i++) {
14            hash_inserir(&tabelaPalavrasReservadas, (string)
                tabelaReservados[i].lex, tabelaReservados[i].tipo);
15        }
16
17        DBG_PRINT("RES_hash_init_%d\n", numPalavrasReservadas);
18
19        // Inicializar hash de símbolos
20        {
21            int numSimbolos = sizeof(simbolos) / sizeof(simbolos[0]);
22            int tamanhoSimbolos = numSimbolos * 2 + 1;
23            hash_criar(&tabelaSimbolos, tamanhoSimbolos);
24            for (int i = 0; i < numSimbolos; i++) {
25                hash_inserir(&tabelaSimbolos, (string)simbolos[i],
                    simbolosType[i]);
26            }
27            DBG_PRINT("SYM_hash_init_%d\n", numSimbolos);
28        }
29    }
30

```

```

31 // libera a tabela hash
32 void liberarTabelaReservadas() {
33     hash_destruir(&tabelaPalavrasReservadas);
34     DBG_PRINT("Tabela_hash_de_palavras_reservadas_liberada\n");
35
36     // Liberar tabela de símbolos
37     hash_destruir(&tabelaSimbolos);
38     DBG_PRINT("Tabela_hash_de_símbolos_liberada\n");
39 }
40
41 //adiciona os tokens que coletamos a tabela com sua tipagem e tudo
    mais
42 int addToken(Token result) {
43     if (tokenCount >= MAX_TOKENS) {
44         printf("Erro: número máximo de tokens atingido.\n");
45         return -1;
46     }
47     DBG_PRINT("Adicionando token: lexema='%s', tipo=%d, linha=%d\n"
48         ,
49         result.lexema, result.tipo, result.linha);
50     tokens[tokenCount] = result; // struct copy
51     tokenCount++;
52     return 0;
53 }

```


4 Conclusão

4.1 Integração dos automatos

Nosso trabalho final resultou em um analisador léxico, que recebe um arquivo .txt, e começa na função main como dito anteriormente, ele analisa primeiro qual automato ele vai utilizar, se vai ser o de símbolos, de números e etc, e logo após coloca o automato em ação, somando a variável token a cada caractere lido

4.2 Desafios Superados

Um dos desafios foi como definir as palavras reservadas e símbolos, sem ter que fazer um por um, vimos uma forma de resolver isso, com uma tabela e uma busca hash, para poder analisar perfeitamente termos específicos

4.3 Resultados alcançados

Saidas geradas, demonstrações de casos teste